Digital Signature Algorithm (DSA)

Aim

To implement DSA, a public-key digital signature scheme that proves authenticity and integrity of a message without encrypting it. DSA uses public parameters `p`, `q`, and `g`; the sender keeps private key `x`, publishes public key `y`, signs each message with a fresh one-time secret `k`, and produces the signature pair `(r, s)`. The receiver verifies the message using only `p`, `q`, `g`, `y`, the message, and `(r, s)`.

Algorithm

DSA uses the original standard terms below: `p` is the prime modulus, `q` is the subgroup prime, `g` is the generator, `x` is the private key, `y` is the public key, `k` is the one-time signing secret, `H` is the message hash, and `(r, s)` is the signature.

```
Function Sign(message, p, q, g, x, k):
    H ← hash(message) mod q
    y ← (g^x) mod p                                          // public key
    r ← ((g^k) mod p) mod q
    k_inv ← k^-1 mod q
    s ← k_inv * (H + x * r) mod q
    if r = 0 or s = 0 then
        choose another k
    return public key y and signature (r, s)
```

```
Function Verify(message, p, q, g, y, r, s):
    if r < 1 or r > q - 1 then
        return invalid
    if s < 1 or s > q - 1 then
        return invalid
    H ← hash(message) mod q
    w ← s^-1 mod q
    u1 ← H * w mod q
    u2 ← r * w mod q
    v ← (((g^u1) * (y^u2)) mod p) mod q
    if v = r then
        return valid
    else
        return invalid
```

Output

Sign — message `hello` with public setup `p = 283`, `q = 47`, and generator seed `h = 60`

The UI first derives the DSA generator `g` from `p`, `q`, and `h`:

Quantity	Calculation	Value
exponent	(p - 1) / q = (283 - 1) / 47	6
g	hˆexponent mod p = 60ˆ6 mod 283	230

The sender uses private seed `12345` and one-time seed `67890`. The UI folds these seeds into DSA's valid range `1` to `q - 1`.

Quantity	Calculation	Value
x	((12345 - 1) mod (47 - 1)) + 1	17
k	((67890 - 1) mod (47 - 1)) + 1	40
H	(104 + 101 + 108 + 108 + 111) mod 47	15
y	gˆx mod p = 230ˆ17 mod 283	225
r	(gˆk mod p) mod q = (230ˆ40 mod 283) mod 47	4
k_inv	kˆ-1 mod q = 40ˆ-1 mod 47	20
s	k_inv * (H + x * r) mod q = 20 * (15 + 17 * 4) mod 47	15

Result: `hello` -> public key `y = 225`, signature `(r = 4, s = 15)`

Verify — received message `hello`, public key `y = 225`, and signature `(r = 4, s = 15)`

The receiver does not know `x` or `k`. It verifies using only public data and the received signature.

Quantity	Calculation	Value
H	(104 + 101 + 108 + 108 + 111) mod 47	15
w	sˆ-1 mod q = 15ˆ-1 mod 47	22
u1	H * w mod q = 15 * 22 mod 47	1
u2	r * w mod q = 4 * 22 mod 47	41
v	((gˆu1 * yˆu2) mod p) mod q = ((230ˆ1 * 225ˆ41) mod 283) mod 47	4
check	v = r, so 4 = 4	valid

Result: received packet -> signature valid

Expansion of original DSA terms

Term	Expansion / meaning
DSA	Digital Signature Algorithm
p	Prime modulus; the main prime used for modular arithmetic
q	Subgroup prime; a prime divisor of p - 1
h	Generator seed used to derive g
g	Generator / signing base used in signing and verification
x	Sender's private key; kept secret
y	Sender's public key; shared with the receiver
k	One-time signing secret; must be fresh for every signature
H	Hash value of the message reduced modulo q
r	First signature part; produced from g, k, p, and q
s	Second signature part; binds H, x, r, and k together
w	Modular inverse of s modulo q
u1	Verification weight for the message hash H
u2	Verification weight for the public key y
v	Recomputed value checked against r
k_inv	Modular inverse of k modulo q